Docker

From “It Works on My Machine” to “It Works Everywhere”

The Problem: Development Environment Hell

```
Engineer A:
  "My Node.js app runs perfectly!"
Engineer B:
  "I get errors when I run your code..."
Client A:
  "It works on macOS but not Windows..."
Client B:
  "My MongoDB version is different..."
Server Manager:
  "It fails behind Nginx..."
```

What's Really Happening

Layer	Varies by machine?	Impact
OS/Kernel	Yes	Path, permissions, sockets
Runtimes	Yes	Node 16 vs 20 behavior
Dependencies	Yes	Native modules, ABI, builds
Services	Yes	Mongo/Nginx versions & config

Tiny differences → **non-reproducible** environments.

Docker — What It Is (Precisely)

- Image: Snapshot (recipe) of an environment (OS libs + runtime + your app)
- Container: A **running** instance of that image (isolated process + filesystem)
- Compose: Orchestrates multiple containers (e.g., web + db + nginx)

Same image ⇒ same bits ⇒ same behavior on **any** host with Docker.

Why Docker Solves It

- Pin versions (Node 22, Mongo 7, Nginx stable) inside images
- Bundle dependencies in the image layer (repeatable builds)
- Isolate each service (no host conflicts/ports/libs)
- Development == Operation parity (run the *same* containers everywhere)

Docker for Software Engineers

1. Instant Setup

```
# Without Docker
brew install node nginx   # and install MongoDB separately

# With Docker
docker compose up
```

2. Perfect Consistency

- Same Node version
- Same DB config
- Same proxy rules/configurations

3. Easy Cleanup

```
docker compose down
```

4. Fast Onboarding

❌ 2 days setup → ✅ 10 minutes

5. Reliable Deployment

Same environment locally & in production.

6. Scalable

Run multiple containers per server via Nginx load balancing.

7. Cost-Efficient

One VPS runs many apps.

Run The previous Node.js app and NGINX using Docker

Three Steps

1. Make app/index.js Node.js application a container
2. Make Nginx reverse proxy server a container.
3. Run these containers

```
docker
├── app
│   └── Dockerfile
├── nginx
│   ├── Dockerfile
│   └── nginx.conf
├── docker-compose.yml
└── run.sh
```

Make app/index.js a container

We make a Dockerfile for the Node.js app: it is an instruction about how to build the isolated environment.

```
FROM node:22-alpine
# Set working directory
WORKDIR /app
# Copy package files
COPY app/package*.json ./
# Install dependencies (production only)
RUN npm ci --omit=dev
# Copy application code
COPY app/ ./
# Expose port 3000
EXPOSE 3000
# Run as the non-root user of the image
USER node
# Start the application
CMD ["npm", "start"]
```

docker-compose.yml

docker-compose.yml is used to orchestrate multiple docker containers.

```
  app:
    build:
      context: ..
      dockerfile: docker/app/Dockerfile
    container_name: nodejs_app
    restart: unless-stopped
    environment:
      - NODE_ENV=production
      - PORT=3000
    expose:
      - "3000"
    networks:
      - app-network
```

The context means the base directory, so dockerfile is located in `../docker/app/Dockerfile`.

Make Nginx a container

The Dockerfile for Nginx server is simple because it copies only the nginx.conf and expose two ports.

```
FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80 443
```

- The host computer uses 8080, but it maps to the container port 80.

```
    ports:
      - "8080:80"  # Host port 8080 → Container port 80
```

The context (base directory) is `./nginx`, so the dockerfile is located in `./nginx/Dockerfile`.

```
  nginx:
    build:
      context: ./nginx
      dockerfile: Dockerfile
    container_name: nginx_proxy
    restart: unless-stopped
    ports:
      - "8080:80"  # Host port 8080 → Container port 80
    depends_on:
      - app
    networks:
      - app-network
```

This is nginx.conf file: it accesses the node.js app at port 3000, and it listens to port 80.

```
# NGINX Configuration for Docker
# Uses container name instead of localhost

upstream nodejs_app {
    # Use Docker service name
    server app:3000;
    keepalive 8;
}

server {
    listen 80;
    server_name localhost;

    location / {
        proxy_pass http://nodejs_app;
        proxy_http_version 1.1;

        # Pass headers
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

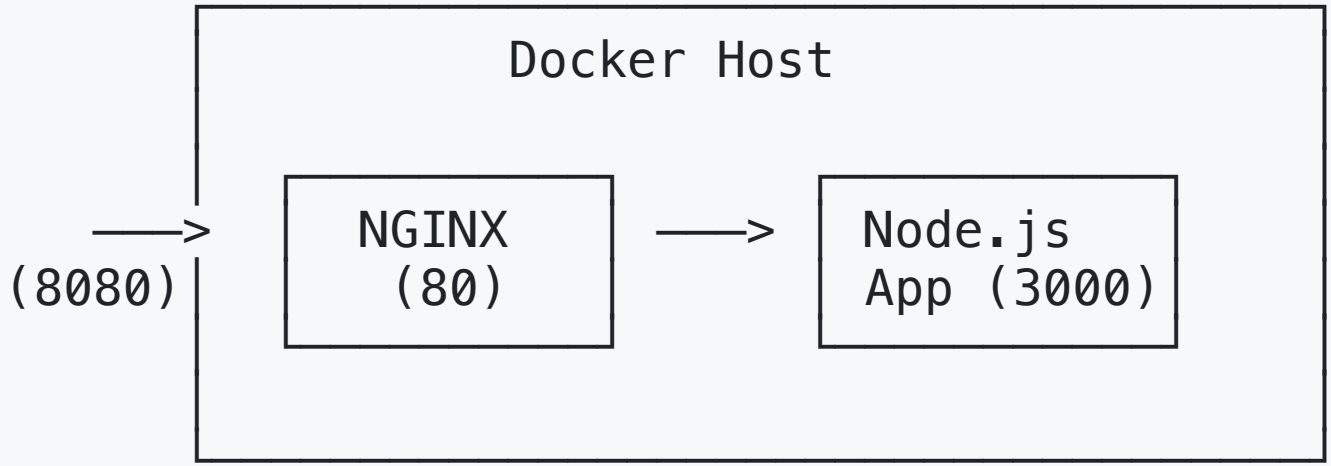
Orchestrate two containers

```
services:
  # Node.js Application
  app:
    ...

  # NGINX Reverse Proxy
  nginx:
    ...

networks:
  app-network:
    driver: bridge
```

This diagram shows how the ports are mapped from the host (8080) to the Nginx container (80) and the node.js (3000).



Run the Docker

Make sure Docker Desktop (or command line tools) are installed and running.

Build the docker containers

```
docker compose build

# or, to build from scratch
docker compose build --no-cache
```

Older Docker installations use the command `docker-compose` instead of `docker compose`.

Run & Shutdown docker containers

```
docker> docker compose up -d
[+] Building 0.0s (0/0)                          docker:desktop-linux
[+] Running 3/3
 ✔ Network docker_app-network  Created                          0.0s
 ✔ Container nodejs_app        Started                          0.0s
 ✔ Container nginx_proxy       Started                          0.0s

docker> docker compose down
[+] Running 3/2
 ✔ Container nginx_proxy       Removed                          0.1s
 ✔ Container nodejs_app        Removed                          0.6s
 ✔ Network docker_app-network  Removed                          0.0s
```

Then, we can access the Docker server with `http://localhost:8080`.

docker_https

We can build the docker that supports https.

Build certificates

Install and run mkcert to make the certificate and the key (in `docker_https/nginx/certs`).

```
brew install mkcert nss
mkcert -install
mkdir -p docker_https/nginx/certs && cd docker_https/nginx/certs
mkcert localhost 127.0.0.1 ::1 myapp.local
```

The script `docker_https/run.sh` creates the certificates automatically if they are missing.

The generated files are in the `certs` directory.

```
docker_https/
├── app
│   └── Dockerfile
├── docker-compose.yml
├── nginx
│   ├── certs
│   │   ├── localhost+3-key.pem
│   │   └── localhost+3.pem
│   ├── Dockerfile
│   └── nginx-https.conf
└── run.sh
```

Update Nginx

The docker file should copy the certs directory to container.

```
FROM nginx:alpine
COPY nginx-https.conf /etc/nginx/conf.d/default.conf
COPY certs/ /etc/nginx/certs/
EXPOSE 80 443
```

1. Redirect the HTTP port (80 in the container, 8080 on the host) to HTTPS (443 in the container, 8443 on the host).
2. Make the HTTPS server block.

```
upstream nodejs_app {
    server app:3000;
    keepalive 8;
}

# HTTP Server - Redirect to HTTPS
server {
    listen 80;
    server_name localhost;

    # Redirect all HTTP to HTTPS
    return 301 https://$host:8443$request_uri;  # the host maps port 8443 to 443
}

# HTTPS Server
server {
    listen 443 ssl;
    server_name localhost;

    # SSL Certificate and Key
    ssl_certificate     /etc/nginx/certs/localhost+3.pem;
    ssl_certificate_key /etc/nginx/certs/localhost+3-key.pem;

    ...
}
```

Update docker-compose.yml

Add 8443 to 443 port mapping.

```
services:
  ...
  # NGINX Reverse Proxy
  nginx:
    ...
    ports:
      - "8080:80"  # Host port 8080 → Container port 80
      - "8443:443"
...
```

Build and run Docker containers

```
docker compose build --no-cache
docker compose up -d
```

Then, we can access the Docker server with `https://localhost:8443`.